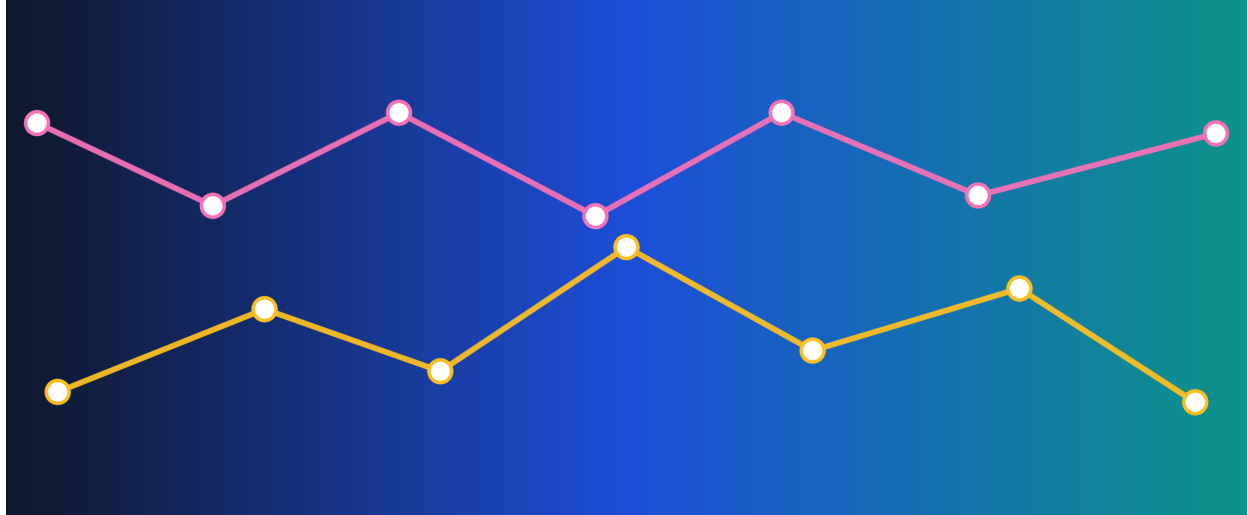




FUNCTIONAL SPECIFICATION DOCUMENT

Smart Metro Ticket Booking System

Digital, Self-Service Metro Ticket Booking & QR-Based Ticketing on ServiceNow

FSD — Functional & Non-Functional Specification

Document Version 1.0
04 August 2026

Source: github.com/JanapatiDivyaSaranya/Metro-Ticket-Generating-System

Table of Contents

PROJECT REPORT FORMAT

1. INTRODUCTION.....	3
1.1 Project Overview.....	3
1.2 Purpose.....	3
2. IDEATION PHASE.....	3
2.1 Problem Statement.....	3
2.2 Empathy Map Canvas.....	4
2.3 Brainstorming.....	5
3. REQUIREMENT ANALYSIS.....	5
3.1 Customer Journey map.....	5
3.2 Solution Requirement.....	6
3.3 Data Flow Diagram.....	8
3.4 Technology Stack.....	9
4. PROJECT DESIGN.....	9
4.1 Problem Solution Fit.....	10
4.2 Proposed Solution.....	10
4.3 Solution Architecture.....	11
5. PROJECT PLANNING & SCHEDULING.....	13
5.1 Project Planning.....	13
6. FUNCTIONAL AND PERFORMANCE TESTING.....	14
6.1 Performance Testing.....	14
7. RESULTS.....	16
7.1 Output Screenshots.....	16
8. ADVANTAGES & DISADVANTAGES.....	18
9. CONCLUSION.....	19
10. FUTURE SCOPE.....	19
11. APPENDIX.....	19
Source Code (if any).....	19
Dataset Link.....	20
GitHub & Project Demo Link.....	20

1. INTRODUCTION

1.1 Project Overview

The Smart Metro Ticket Booking System digitizes metro ticket booking end-to-end on the ServiceNow Now Platform. Passengers select their source and destination stations, passenger type, and number of tickets through a Service Catalog form; the platform automatically calculates the fare, processes digital payment, and issues a unique QR-coded digital ticket instantly. The solution is powered by Service Catalog, Flow Designer, Business Rules, Integration Hub, and Notifications, and serves four personas: Passengers, Station Managers, the Metro Operations Team, and IT Admins.

This removes the dependency on staffed counters and token machines that currently define metro ticketing — replacing queues, manual fare calculation, and paper tickets with a self-service, auditable, digital-first experience.

1.2 Purpose

This Functional Specification Document (FSD) formally defines what the Smart Metro Ticket Booking System does, who it is for, and how its components fit together. It translates the ideation output — problem statement, empathy map, and brainstorming — into concrete functional and non-functional requirements, documents the solution architecture and technology stack on ServiceNow, and records the project planning, testing approach, and results. It is intended for reviewers, evaluators, and any developer who needs to build on, maintain, or extend the application.

2. IDEATION PHASE

2.1 Problem Statement

Among urban public transport options, the metro is one of the most heavily used, prized for its speed, affordability, and role in easing road traffic. Even so, buying a metro ticket in many networks still relies mainly on staffed counters, coin-operated token machines, or kiosks that are only partially digitised. A rider typically has to go to the station in person, wait in line, pick a source and destination, hand over cash or a card, and walk away with a paper ticket or token before being let onto the platform.

“Metro passengers regularly deal with long counter queues, fare mistakes from manual calculation, restricted counter hours, and reliance on paper tickets or tokens — all of which add delay, crowd the ticketing area, and make travel less pleasant, particularly during rush hour or for first-time and differently-abled riders.”

Problem Statement Canvas (Who / What / Why)

Element	Description
I am (User)	An everyday commuter — office-goer, student, occasional rider, senior citizen, or a differently-abled passenger — who relies on the metro.

Element	Description
Trying to	Get a metro ticket booked fast for a chosen source and destination, skipping the wait in line.
But I am stuck	Waiting at busy ticket counters or dealing with vending machines that are sluggish, offline, or hard to use.
Because	Ticket issuing is mostly a manual job, counters are short-staffed at busy times, and fares are worked out by hand.

Key Points

- Ticket-counter lines stretch long during office rush, exam season, and festival travel.
- Working out fares by hand is slow and leaves room for mistakes.
- Physical counters keep limited hours even though travel demand runs all day.
- A lost or damaged token/paper ticket means extra time re-verifying at the counter.
- Elderly, disabled, and visually impaired riders get little dedicated help at manual counters.
- There's no single digital record of a rider's past trips or spending.
- Running counters, staffing them, and stocking tokens is costly for the metro authority.

A self-service digital booking system is clearly needed — one where a rider picks their stations, sees the fare worked out automatically, and gets a QR-based digital ticket right away, with no counter visit involved. Delivering this on ServiceNow means the solution can draw on the Service Catalog for a guided booking flow, Flow Designer to automate fare calculation and ticket issuance, and Notifications to deliver the digital ticket instantly.

2.2 Empathy Map Canvas

Persona: Arjun Mehta, 29, an employee at a private firm, whose goal is to get to the office on time every day by metro without losing time at the ticket counter.



Fig 2.2: Empathy Map Canvas for the daily metro commuter persona

Insight Summary

The empathy map makes clear that the biggest source of frustration for commuters is time lost to manual, in-person steps and not knowing the fare or queue length in advance. What riders are really after is a quick, self-service way to book and pay before they even reach the station, ideally from their phone — a direct argument for a ServiceNow Service Catalog-driven booking flow that automates the fare and hands over an instant QR ticket.

2.3 Brainstorming

The team held a brainstorming session to surface possible ways to solve the manual, queue-driven metro ticketing problem. Ideas were clustered by theme, then ranked using an Impact-vs-Effort prioritization matrix.

Idea Grouping (Affinity Mapping)

Theme	Grouped Ideas
Core Booking Experience	Service Catalog booking form, station/date/passenger selection
Communication & Delivery	Notifications (SMS/Email/Push), booking confirmation
Payments	UPI/Card/Wallet integration, season pass renewal
Operations & Admin	Admin dashboard, analytics/reporting, gate hardware integration
Support & Accessibility	Virtual Agent/chatbot, voice-assisted accessibility mode, cancellation

Finalized MVP Scope

- A Service Catalog item to pick source station, destination station, travel date, and passenger count.
- A Flow Designer workflow to work out the fare automatically from station distance/zone and passenger count.
- Automatic issuing of a unique digital ticket with an embedded QR code once booking succeeds.
- A notification step to send the digital ticket and confirmation to the commuter right away.
- Basic tracking of each user's booking history/status via the Service Catalog request record.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

The journey tracks every touchpoint from first hearing about the digital option to giving feedback after the trip, including the current pain point in the manual process and the digital opportunity ServiceNow opens up at each stage.

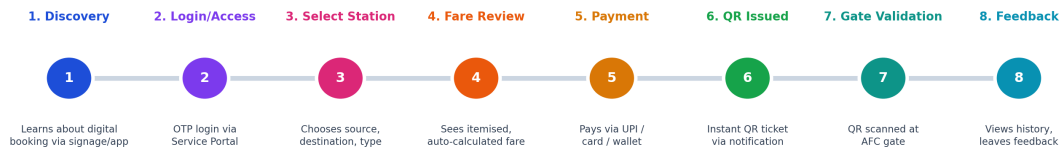


Fig 3.1: Passenger journey from discovery to post-trip feedback

Stage	Current Pain Point (Manual)	Digital Opportunity (ServiceNow)
1. Discovery	No clear sign a fast digital option exists; long lines during busy hours.	In-app banner, portal landing page, WhatsApp entry point.
2. Login/Access	Manual counters need cash and a physical visit; no saved history.	OTP-based single sign-on via Service Portal in under 30 seconds.
3. Select Station	Station-name mix-ups common at counters.	Catalog client scripts auto-validate routes, block bad combinations.
4. Fare Review	Fare disputes common due to unclear/manual pricing.	Automated fare-logic script gives consistent, auditable pricing.
5. Payment	Cash-only counters slow throughput.	UPI, card, wallet via a Flow Designer payment sub-flow.
6. QR Ticket Generated	Paper tickets get lost or damaged; reprint means another line.	QR ticket lives digitally, always retrievable, no paper.
7. Travel & Gate Validation	Manual ticket checks are slow, often disputed.	Instant QR validation; Station Manager sees live status.
8. Feedback / History	No structured channel for feedback or travel history today.	Digital trail supports data-driven route/peak-time planning.

Key Takeaways

- What used to be a multi-step manual process — queue, cash, paper ticket, manual gate check — shrinks to a self-service digital flow finished in under two minutes.
- Every stage past login is now instrumented, giving Metro Operations end-to-end visibility that doesn't exist with paper tickets today.
- Station Managers and IT Admins are stakeholders in this journey too — the former for live gate validation, the latter for configuring and supporting the platform.

3.2 Solution Requirement

Data Flow & User Stories (context)

Four human actors and two external systems exchange data with the core booking platform: Passengers kick off booking requests; Station Managers keep station and fare master data current; the Metro Operations Team consumes operational reports; IT Admins handle configuration and access; the Payment Gateway processes transactions; and the Notification Channel delivers tickets and alerts. The system itself splits into six core processes — Station &

Route Selection, Fare Calculation, Payment Processing, QR Ticket Generation, Notification Dispatch, and Reporting & Analytics.

Functional Requirements

ID	User Story	Acceptance Criteria
US-01	As a passenger, I want to pick source/destination from a searchable catalog form, so I can book without a counter visit.	Station list loads from Station Master; invalid pairs blocked; search-as-you-type supported.
US-02	As a passenger, I want to see the calculated fare right after entering travel details, so I know the exact amount before paying.	Fare updates live as station/type/count changes; Student/Senior discounts apply automatically.
US-03	As a passenger, I want to pay digitally via UPI, card, or wallet, so exact cash isn't required.	Payment sub-flow supports 3+ digital modes; failed payments offer a clear retry path.
US-04	As a passenger, I want my ticket as a QR code right after payment, so I can travel without a printed ticket.	QR generated within seconds of payment; delivered via app notification and email/WhatsApp.
US-05	As a passenger, I want to view and re-download past bookings, so I have a record for reimbursement or reuse.	Booking history available in the Service Portal, filterable by date and route.
US-06	As a station manager, I want to maintain station details and fare zones, so booking data stays accurate.	Station Master records editable via a controlled table with approval workflow for changes.
US-07	As a station manager, I want to validate QR tickets at the gate in real time, so only paid passengers get through.	QR scan returns valid/invalid/used status within 2 seconds; duplicate scans flagged.
US-08	As a station manager, I want a dashboard of today's ticket volume at my station, so I can plan staffing.	Dashboard refreshes in near real time and is filterable by hour.
US-09	As an operations manager, I want consolidated analytics on peak travel times and popular routes, so I can plan schedules.	Reports/Performance Analytics dashboards aggregate booking data by route, time band, type.
US-10	As an operations manager, I want visibility into digital-vs-cash revenue, so I can track digital adoption.	Revenue split report available monthly and exportable.
US-11	As an operations manager, I want automated alerts for failed or disputed payments, so I can act proactively.	Flow Designer raises an incident/case record for any exception.
US-12	As an IT admin, I want to configure catalog variables and fare rules without code changes, so business changes ship quickly.	Fare Rule Table and catalog variables are admin-configurable via the ServiceNow UI.
US-13	As an IT admin, I want role-based access across passenger, station manager, and operations views, so data stays visible only to authorized users.	ACLs restrict table and UI Page access strictly by assigned role.
US-14	As an IT admin, I want monitoring and logs for the QR and payment integrations, so I can troubleshoot failures quickly.	Integration Hub spokes log request/response with correlation IDs for traceability.

Non-Functional Requirements

Category	Requirement
Performance	Fare calculation and QR generation complete within 3 seconds of payment confirmation under normal load.
Scalability	The solution handles concurrent booking spikes during peak commute hours without degrading.
Availability	Core booking and QR validation target 99.5% uptime during metro operating hours.
Security	All payment data goes through PCI-compliant gateways; ACLs govern every sensitive table.
Usability	A first-time user can complete booking in under 5 steps, with accessible, mobile-friendly UI Pages.
Auditability	Every booking, payment, and QR generation event is logged with a timestamp and correlation ID.
Maintainability	Fare rules and station data are configurable by IT Admins without a code deployment.

ServiceNow Component Mapping

Requirement Area	ServiceNow Component	Purpose
Ticket Booking Form	Service Catalog + Catalog Client Scripts	Captures Source, Destination, Passenger Type, Number of Tickets with real-time validation.
Fare Calculation	Business Rules / Script Includes	Runs fare logic server-side based on station distance and passenger type.
Booking-to-Ticket Workflow	Flow Designer	Orchestrates payment trigger, QR generation, notification dispatch.
Payment Integration	Integration Hub / REST Spokes	Connects to external payment gateway APIs for UPI, card, and wallet.
QR Code Generation	Integration Hub / Script Include	Generates a unique QR image and links it to the ticket record.
Alerts & Ticket Delivery	Notifications (Email/SMS/Push)	Delivers digital tickets and status alerts to passengers and staff.
Passenger Self-Service	Service Portal / UI Pages	Hosts the booking form, fare preview, booking history, QR display.
Reporting	Performance Analytics / Reports	Gives Metro Operations usage, revenue, and peak-time dashboards.

3.3 Data Flow Diagram

Level 0 (Context): the diagram below identifies the four human actors and two external systems exchanging data with the core booking platform.

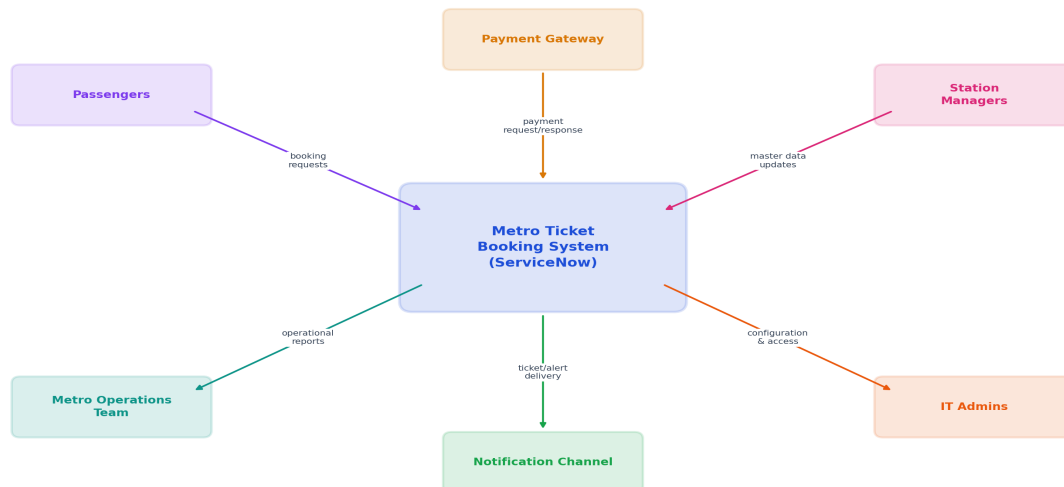


Fig 3.3: Data Flow Diagram — Level 0 Context view

Level 1 (Process Decomposition): the system splits into six core processes — Station & Route Selection, Fare Calculation, Payment Processing, QR Ticket Generation, Notification Dispatch, and Reporting & Analytics — mapping directly onto Flow Designer sub-flows and Script Includes, reading from and writing to three core data stores: Station Master, Fare Rule Table, and Ticket/QR Records.

3.4 Technology Stack

Layer	Technology / Component	Purpose
Presentation	Service Portal, UI Pages, WhatsApp Business Chatbot	Passenger-facing booking form, fare preview, ticket display, chat-based booking.
Application / Process	Service Catalog, Flow Designer, Business Rules, Script Includes, Catalog Client Scripts	Captures booking input, orchestrates the booking-to-ticket workflow, executes fare logic.
Integration	Integration Hub, REST/SOAP APIs, Payment Gateway, QR Code Generation Library	Connects the platform to external payment and QR services with logged, traceable calls.
Data	Custom Tables — Station Master, Fare Rule Table, Ticket/QR Records, Transaction Logs	Stores reference and transactional data needed for booking, validation, reporting.
Notification	Email, SMS Gateway, Push Notifications, WhatsApp Business API	Delivers QR tickets, payment confirmations, and exception alerts.
Reporting & Analytics	Performance Analytics, Reports, Dashboards	Surfaces booking volume, revenue split, and peak-time insights.
Security & Administration	ACLs, OAuth 2.0, Data Encryption, Role-based Views	Restricts data/functionality by persona and secures payment/QR data end-to-end.

4. PROJECT DESIGN

4.1 Problem Solution Fit

Today, buying a metro ticket depends heavily on physical counters and handling cash. Passengers deal with long queues, limited digital payment acceptance, and no way to look back at past bookings. Station Managers update

fare and station information by hand. Metro Operations has no consolidated view of ticket volume, revenue, or peak-time demand. IT Admins can't change fare rules or catalog configuration without a development request.

Persona	Problem / Pain Point	Solution Fit
Passengers	Long queues, cash-only counters, no digital ticket or booking history.	Self-service Service Catalog booking with instant QR delivery and viewable history.
Station Managers	Manual station/fare updates and slow, unverifiable gate checks.	Controlled, approval-based master-data workflow plus real-time QR validation (<2s).
Metro Operations Team	No consolidated view of ticket volume, revenue, or peak patterns.	Performance Analytics dashboards on usage, revenue split, and peak-time trends.
IT Admins	Fare/catalog changes require code deployment, slowing updates.	No-code configuration of catalog variables, fare rules, and ACLs in the ServiceNow UI.

Why Now (Market & Timing Fit)

- Rising smartphone and UPI/digital-wallet adoption means most passengers already have what they need to pay and receive tickets digitally.
- Post-pandemic commuter expectations favour contactless, counter-free ticketing over handling cash.
- The metro network is already planning QR-based Automated Fare Collection (AFC) gate hardware, making this the natural point to digitize the booking front end.

4.2 Proposed Solution

The proposed solution is a ServiceNow-based Smart Metro Ticket Booking system that lets passengers look up routes, check fares, pay digitally, and get a QR ticket within minutes — while giving Station Managers, Metro Operations, and IT Admins the tools to manage stations, track performance, and configure the platform without counter dependency or custom code. A passenger picks a source and destination from a searchable catalog form, sees an instant itemized fare with passenger-type discounts applied, pays via UPI/card/wallet, and receives a unique, single-use QR ticket by app notification, email, and WhatsApp/SMS. That same QR code is validated at the station gate in real time, with every step logged for audit and reporting.

Value Proposition

Persona	Value Delivered
Passengers	Book and travel in under 5 steps, with no queue or cash needed.
Station Managers	Faster, verifiable gate checks and less manual data upkeep.
Metro Operations	Data-driven scheduling and revenue tracking, updated in near real time.
IT Admins	Faster turnaround on fare and configuration changes, with full traceability.

4.3 Solution Architecture

Passengers, Station Managers, Metro Operations, and IT Admins all reach the system through the same Service Portal / UI Pages layer. Requests are handled by the application layer — Service Catalog, Flow Designer, Business Rules / Script Includes — before being routed through the Integration Hub to external systems: the payment

gateway, the QR generation library, and the notification channel. All data lives in the data layer, and Performance Analytics turns this into operational dashboards. ACLs, OAuth 2.0, and data encryption run across every layer.

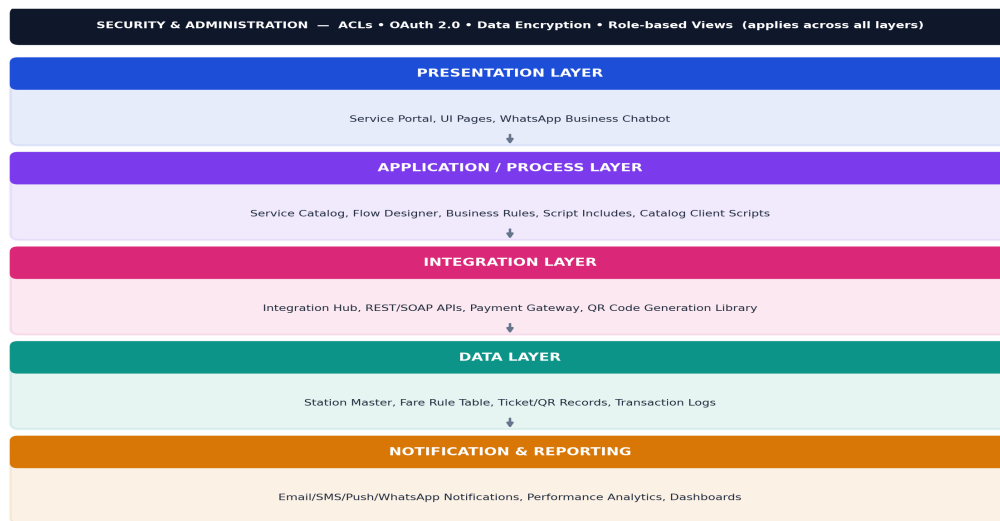


Fig 4.3: Layered solution architecture on the ServiceNow Now Platform

Design Principles

- Single platform — every persona is served from one centrally administered Now Platform instance.
- Low-code first — Flow Designer, Business Rules, and Service Catalog are favoured over custom scripting wherever feasible.
- Traceability by default — every booking, payment, and QR event carries a correlation ID.
- Security by design — ACLs and encryption apply uniformly across presentation, application, and data layers.
- Loose coupling to external systems — the Integration Hub shields the core platform from changes in the payment gateway or QR provider.

Fare Calculation & QR Lifecycle Logic

Fare calculation triggers once a passenger submits Source, Destination, and Passenger Type. The base fare is looked up from the Zone Fare Matrix using source/destination distance; Student and Senior Citizen concessions apply an approved discount percentage; General fare applies the full standard rate; multi-ticket bookings multiply the per-ticket fare by count; and a Source = Destination selection is rejected with a re-selection prompt.

Once payment is confirmed, the QR ticket moves through a defined lifecycle from generation to gate validation and trip closure, with duplicate or expired scans rejected automatically:

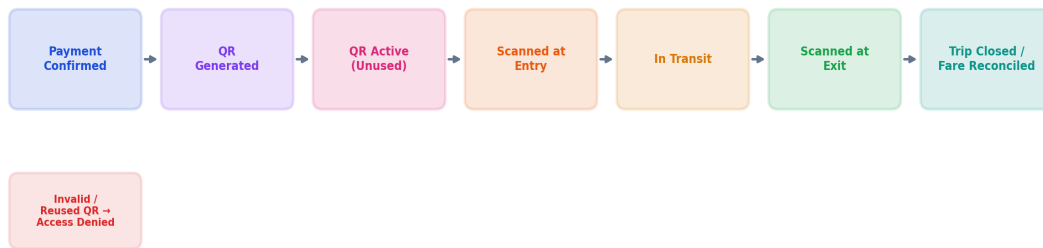


Fig 4.3(a): QR Ticket generation & validation state lifecycle

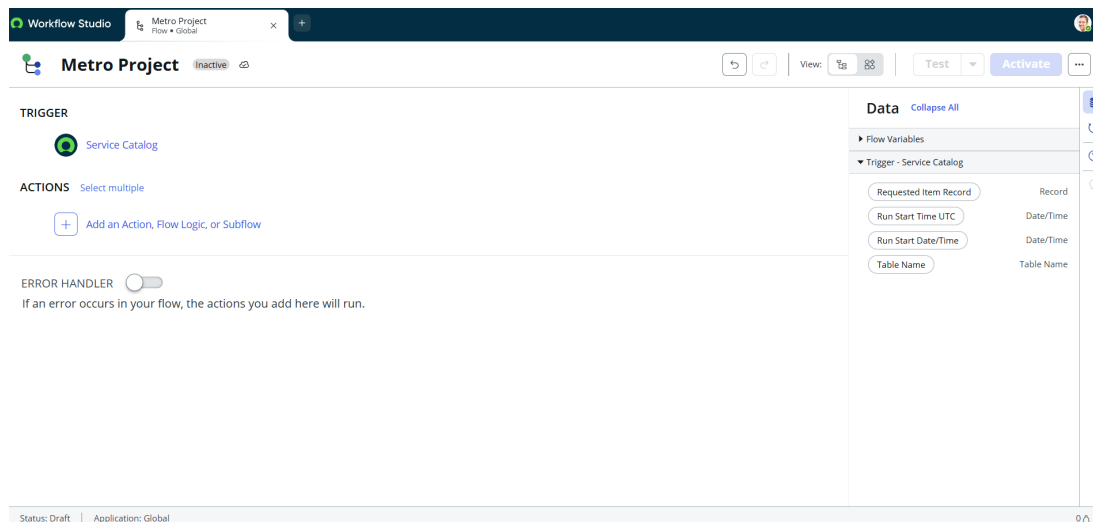


Fig 4.3(b): “Metro Project” flow trigger configured in Flow Designer (Service Catalog trigger)

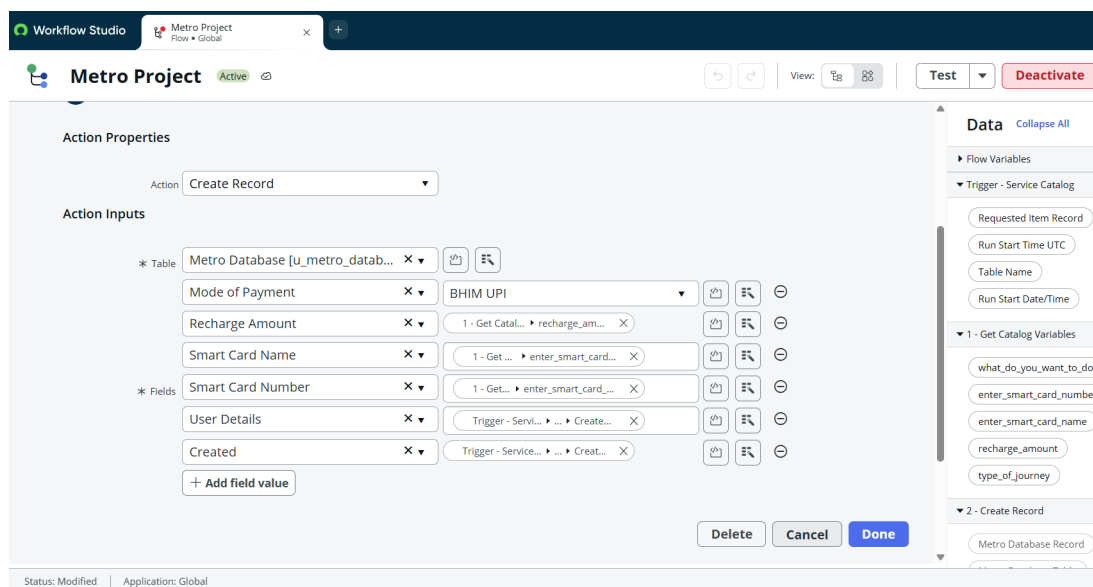


Fig 4.3(c): Flow Designer “Create Record” action writing to the Metro Database table

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The planning phase covers full delivery of the ServiceNow-based metro ticketing solution, from catalog configuration through go-live, split into four work streams: Requirements & Design, Development, Integration, and Testing & Deployment.

Delivery Milestones

Milestone	Description	Owner
Catalog Form Setup	Configure Service Catalog item with booking variables and validations.	IT Admin / ServiceNow Dev
Fare Logic Script	Build fare calculation business rules and concession logic.	ServiceNow Developer
QR Code Integration	Integrate QR generation API into the post-payment workflow.	Integration Developer
Station Details Mapping	Set up station master data, routes, and fare slabs.	Metro Operations Team
UI Pages	Build passenger-facing booking and ticket display screens.	UI/UX Developer
UAT & Deployment	User acceptance testing and production rollout.	QA Team / Project Manager

RACI Matrix

Milestone	Passengers	Station Mgr	Ops Team	IT Admin
Catalog Form Setup	I	I	C	R/A
Fare Logic Script	I	I	C	R/A
QR Code Integration	I	I	I	R/A
Station Details Mapping	I	C	R/A	C
UI Pages	C	I	I	R/A
UAT & Deployment	C	C	A	R

R = Responsible, A = Accountable, C = Consulted, I = Informed.

Risk Management Plan

Risk	Impact	Mitigation	
Delay in payment gateway API access/credentials	High	Initiate vendor onboarding in parallel with Catalog Form Setup.	
Incomplete or inaccurate station master data	Medium	Validate station and fare data with Operations Team before Fare Logic Script begins.	

Risk	Impact	Mitigation	
QR code duplication or security gaps	High	Use signed/encrypted QR tokens and test tamper scenarios during UAT.	
Low passenger adoption of digital channels	Medium	Run a pilot at select stations with in-person support before full rollout.	
AFC gate hardware compatibility issues	Medium	Conduct early hardware integration testing during the QR Code Integration milestone.	
6. FUNCTIONAL AND PERFORMANCE TESTING			

6.1 Performance Testing

UAT Test Scenarios

Scenario ID	Scenario Description	Related Milestone
UAT-SC-01	Passenger books a ticket for a single trip using General passenger type.	Catalog Form Setup
UAT-SC-02	Passenger books multiple tickets in a single transaction.	Catalog Form Setup
UAT-SC-03	System applies Student/Senior Citizen concession correctly.	Fare Logic Script
UAT-SC-04	Passenger completes payment via UPI, card, and wallet.	Fare Logic Script
UAT-SC-05	QR code is generated and delivered after successful payment.	QR Code Integration
UAT-SC-06	QR code is validated correctly at AFC gate for entry and exit.	QR Code Integration
UAT-SC-07	Station Manager views real-time bookings for assigned station.	Station Details Mapping

Sample UAT Test Cases

ID	Test Steps	Expected Result
TC-01	Open catalog form; select Source, Destination, Passenger Type = General, Tickets = 1; submit.	Fare is displayed correctly and booking proceeds to payment.
TC-02	Repeat booking with Passenger Type = Student.	Discounted fare is applied automatically per the concession policy.
TC-03	Complete payment using UPI on a successful test transaction.	Payment confirmed and QR code generated within 5 seconds.
TC-04	Scan generated QR at AFC entry gate simulator.	Gate validates QR as valid, logs entry; status updates to 'Scanned at Entry'.
TC-05	Attempt to reuse the same QR code for a second entry.	System rejects the scan as invalid/reused and logs the attempt.
TC-06	Enter Source and Destination as the same station.	System blocks submission with a validation message.

UAT Entry & Exit Criteria

- Entry: SIT complete with no open Critical/High defects; UAT environment configured with representative data; test cases approved by stakeholders.
- Exit: All planned UAT test cases executed; no open Critical/High defects; formal sign-off from all four stakeholder groups.

Performance Testing — Metrics & Targets

Metric	Target
Catalog form load time	Within 3 seconds under normal load.
Fare calculation response time	Within 2 seconds of source/destination selection.
Payment processing time	Within 5 seconds for UPI/card/wallet transactions.
QR code generation time	Within 5 seconds of payment confirmation.
AFC gate validation time	Within 1.5 seconds per scan.
System availability	99.5% uptime during operational hours.
Error rate under peak load	Less than 1% failed transactions during peak load testing.

Performance Test Types & Scenarios

Scenario ID	Description	Test Type
PERF-SC-01	500 concurrent passengers submitting catalog booking requests during peak hour.	Load
PERF-SC-02	Simultaneous fare calculation requests across multiple stations and zones.	Load
PERF-SC-03	Concurrent payment transactions across UPI, card, and wallet gateways.	Load
PERF-SC-04	Bulk QR code generation immediately following a wave of successful payments.	Load / Spike
PERF-SC-05	Sustained booking traffic over an 8-hour operational window.	Soak
PERF-SC-06	Booking volume increased 2x beyond expected peak to identify breaking point.	Stress
PERF-SC-07	Sudden spike in bookings during a promotional fare offer.	Spike

New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the "Isolate script" field. To disable this feature for all new globally-scoped client-side scripts set the system property "glide.script.block.client.globals" to false.

Name: **fieldValidation**

Applies to: **A Catalog Item**

Active: ☒

UI Type: **All**

Application: **Global**

Type: **onChange**

Catalog item: **Book A Metro Ticket Item**

Variable name: **no_of_passengers**

Applies on a Catalog Item view: ☒

Applies on Requested Items: ☒

Applies on Catalog Tasks: ☒

Script:

```

1 function onChange(control, oldValue, newValue, isLoading) {
2
3   if (isLoading || newValue == '') {
4     return;
5   }
6
7   if (isNaN(newValue)) {
8     alert('please enter the values in number format only');
9   }
10
11
12
13 }

```

Fig 6.1(a): fieldValidation catalog client script enforcing numeric input

Catalog UI policies are similar to standard UI policies. Catalog UI policies dynamically change variables that are part of a catalog item or change how variable sets are handled. Policies can also be applied when the variables are present in a Requested Item or Catalog Task form. [More Info](#)

Applies to: **A Catalog Item**

* Catalog item: **Book A Metro Ticket Item**

* Short description: **Fields Visibility**

Application: **Global**

Active: ☒

When to Apply: **Script**

Catalog UI policy actions are applied only if all the following conditions are met:

1. The catalog UI policy is **Active**
2. The items in the **Conditions** field evaluate to true
3. The field specified in the catalog UI policy is present on the specified catalog item

Catalog Conditions: **mode_of_payment** is **Credit Card**

Applies on a Catalog Item view: ☒

Applies on Catalog Tasks: ☐

On load: ☒

Apply the catalog UI policy actions when the form is loaded or when the user changes values on the form

Fig 6.1(b): Catalog UI Policy conditions used during validation testing

7. RESULTS

7.1 Output Screenshots

Fig 7.1(a): “Book A Metro Ticket Item” catalog form — recharge / booking selection

Fig 7.1(b): Booking form with source, destination, and journey type selected

Fig 7.1(c): QR code modal generated as the digital ticket

Home > My Request - RITM0010001

Activity Attachments Additional Details

What do you want to do Today?
Book a Metro Ticket

Starting From
Ameerpet

Going To
LB Nagar

No of Passengers
1

Type of Journey
Single Journey

Amount for Single Journey
50

Fig 7.1(d): Submitted request (RITM) showing booking details

Metro Databases						
User Details						
	User Details	Smart Card Number	Smart Card Name	Recharge Amount	Mode of Payment	Created
	System Administrator	HYD123GY	PAVANI	10,000	BHIM UPI	2026-07-25 08:48:26
	System Administrator	HYD1234HY	ASHOK	52,000	BHIM UPI	2026-07-25 08:42:58
	System Administrator	HYD1234GT	GANESH	15,200	BHIM UPI	2026-07-25 08:57:58

Fig 7.1(e): Metro Database table with recorded transactions

8. ADVANTAGES & DISADVANTAGES

Advantages

- Eliminates counter queues — booking and recharge available anytime from the Service Portal or WhatsApp.
- Automated, auditable fare calculation removes manual pricing errors and disputes.
- Instant QR-based digital ticket removes dependency on paper tokens/tickets.
- Multi-channel notification (in-app, email, WhatsApp/SMS) ensures reliable ticket delivery.
- Built on a single, centrally administered ServiceNow instance serving all four personas — no separate backend needed.
- Every booking, payment, and QR event is logged with a correlation ID for full traceability.
- No-code configuration means IT Admins can update fare rules and catalog variables without a development cycle.

Disadvantages

- Depends on an active ServiceNow instance, smartphone/WhatsApp access, and network connectivity — no offline mode.
- Relies on third-party Payment Gateway and QR generation vendors being available and performant.
- AFC gate hardware integration is an assumption/constraint rather than a delivered component in this phase.

- High-volume peak-hour usage requires dedicated performance testing (load/stress/soak/spike) before go-live confidence is established.
- Station and fare master data accuracy depends on manual upkeep by the Metro Operations Team.

9. CONCLUSION

The Smart Metro Ticket Booking System demonstrates that a complete self-service ticketing workflow — station selection, automated fare calculation, digital payment, and QR ticket issuance — can be delivered on the ServiceNow platform using Service Catalog, Flow Designer, Business Rules, Integration Hub, and Notifications, serving Passengers, Station Managers, Metro Operations, and IT Admins from a single governed platform. The solution was shaped through a structured ideation and requirement-analysis process — problem statement, empathy mapping, prioritized brainstorming, customer journey mapping, and detailed user stories — and validated through a defined UAT and performance testing approach against explicit response-time and availability targets. It directly addresses the original problem of counter-based queuing, manual fare errors, and paper tickets by giving commuters a fast, auditable, digital-first alternative.

10. FUTURE SCOPE

- Roll out season pass booking with auto-renewal reminders (identified as a P2 “Could Have” in prioritization).
- Add a Virtual Agent / chatbot for booking support and common questions.
- Integrate live IoT/gate-hardware scanning for fully automated AFC validation.
- Introduce a voice-assisted accessibility mode for elderly and differently-abled riders.
- Build out analytics and peak-hour fare prediction (flagged as P3 “Future Scope” in the prioritization matrix).
- Extend group/corporate booking support beyond the current per-passenger flow.
- Expand self-service cancellation and refund workflows for booked tickets.

11. APPENDIX

11.1 Source Code (if any)

fieldValidation — Catalog Client Script (onChange, applies to No of Passengers)

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') { return; }
    if (isNaN(newValue)) {
        alert('Please enter the values in number format only');
    }
}
```

Catalog UI Policy — “Fields Visibility” (condition-driven field control)

Applies to: A Catalog Item | Catalog item: Book A Metro Ticket Item
 Condition: mode_of_payment is Credit Card

Action: enter_payment_mode -> Mandatory: True, Visible: True (on load)

11.2 Dataset Link

Table	Key Fields
Metro Station's Details (u_metro_station_s_details)	Station Name, Station Code, Active
Metro Database (u_metro_database)	User Details, Smart Card Number, Smart Card Name, Recharge Amount, Mode of Payment, Created

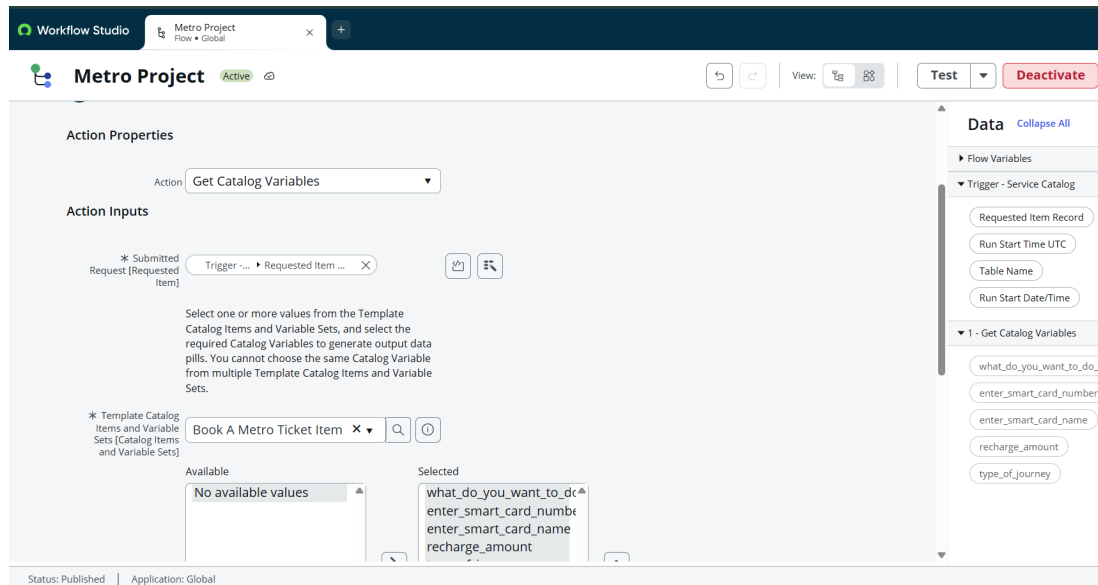


Fig 11.2: Flow Designer “Get Catalog Variables” action reading the Book A Metro Ticket Item template

11.3 GitHub & Project Demo Link

GitHub Repository: <https://github.com/JanapatiDivyaSaranya/Metro-Ticket-Generating-System.git>

Project Demo Link: <https://1drv.ms/v/c/487a17a63a2d0c38/IQB9prtBgVG3SoDuD3u8CbMtAdEMzjyk9y-dGTzqexjS5Sc?e=zYGIUj>